PONTIFICIA UNIVERSIDAD CATÓLICA DEL PERU
UNIVERSIDAD PERUANA CAYETANO HEREDIA

FACULTAD DE CIENCIAS E INGENIERÍA
INGENIERÍA BIOMÉDICA



Entregable N°10: Implementación

Autores:

Arana Ocros, Christian Jesus

Auqui Jota, Luciano Mateo Alejandro

Bastidas Reyes, Mariangel Beatriz

Bravo Larico, Juan Pablo Rai

Crisologo Ramirez, Renato Del Piero

Huidobro Kokubun, Miranda Naomi

Asesores:

Zuñiga Mamani, Juan Manuel

Venancio, Julissa

Nuñez, Isabel

Med. Lazarte, Rodney

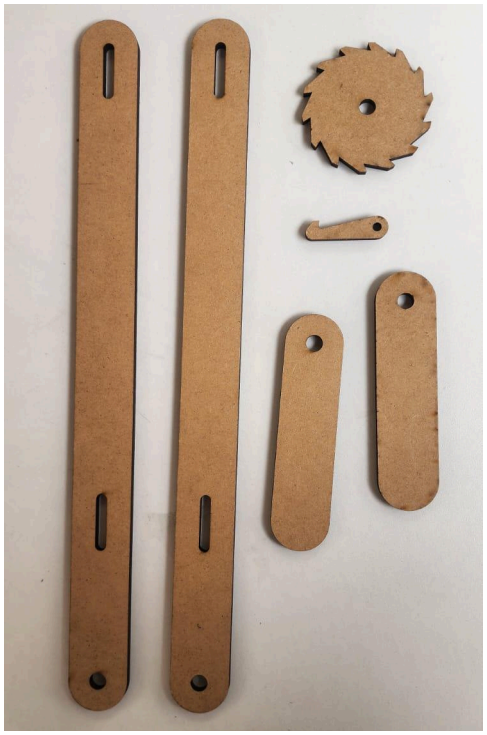
Curso:

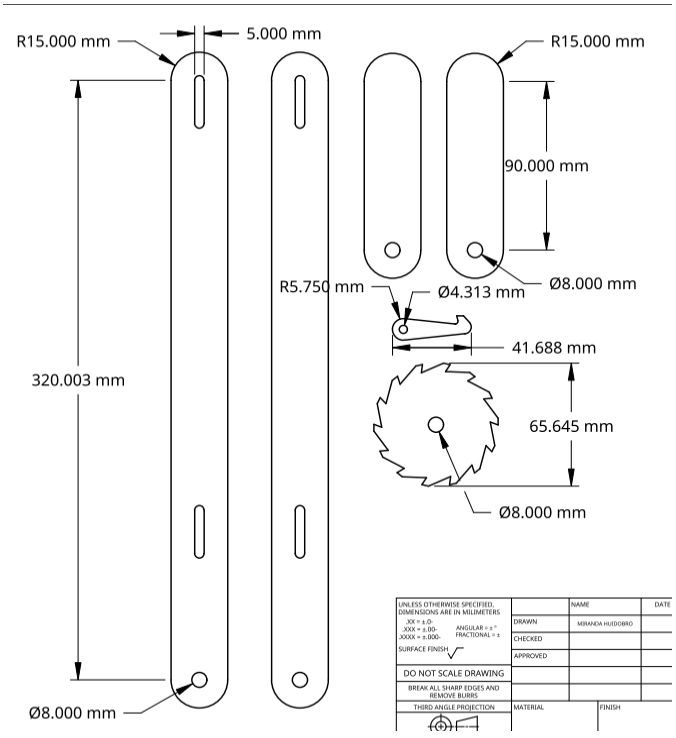
Fundamentos de Biodiseño

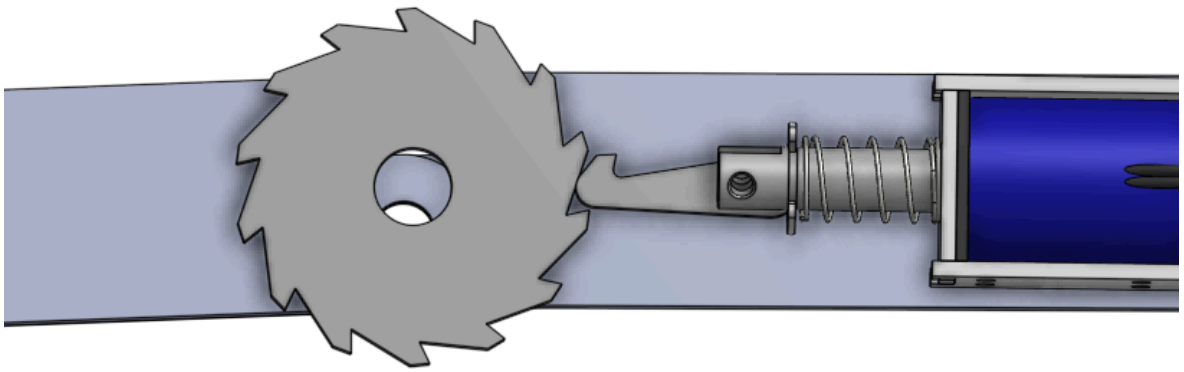
2026

1. Implementación preliminar de los subsistemas según el diseño

1.1. Mecánica y fabricación

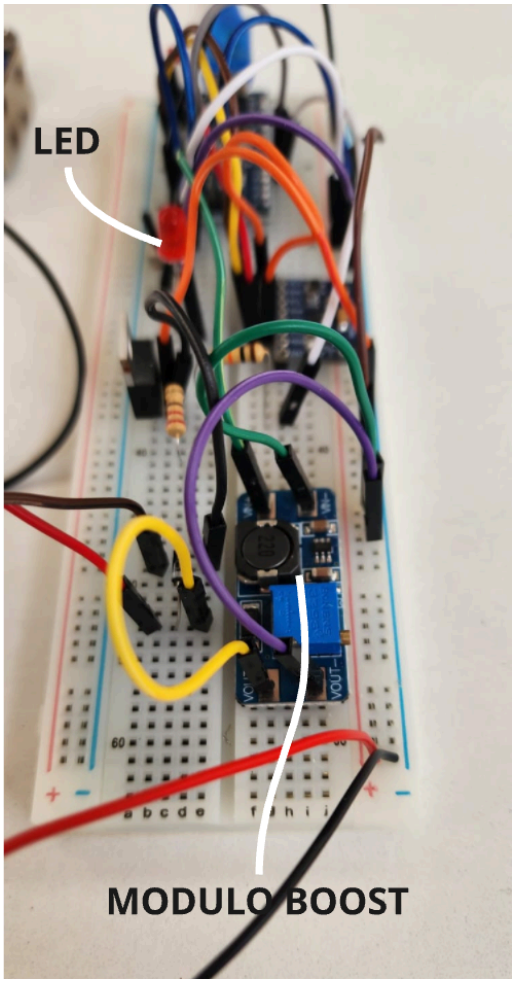


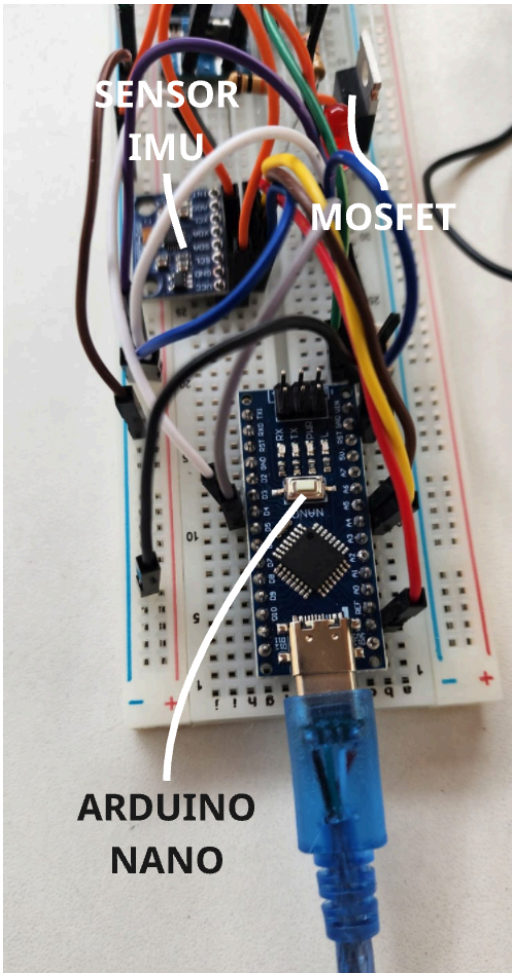


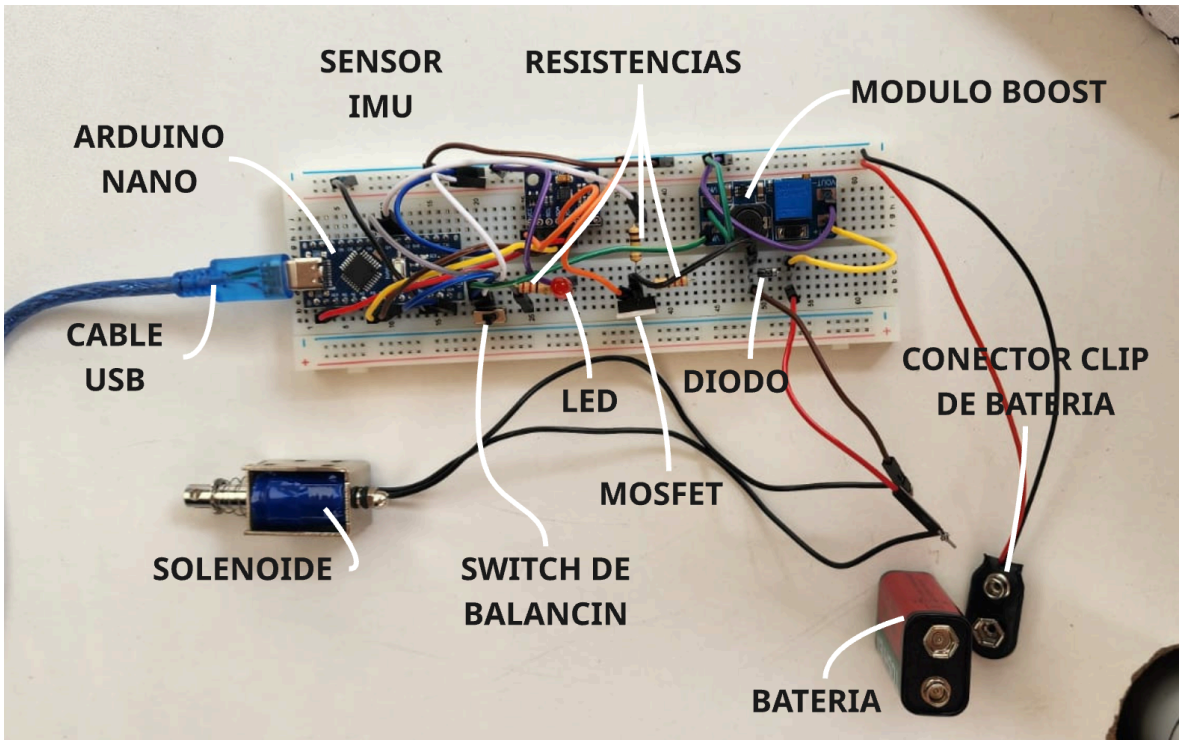


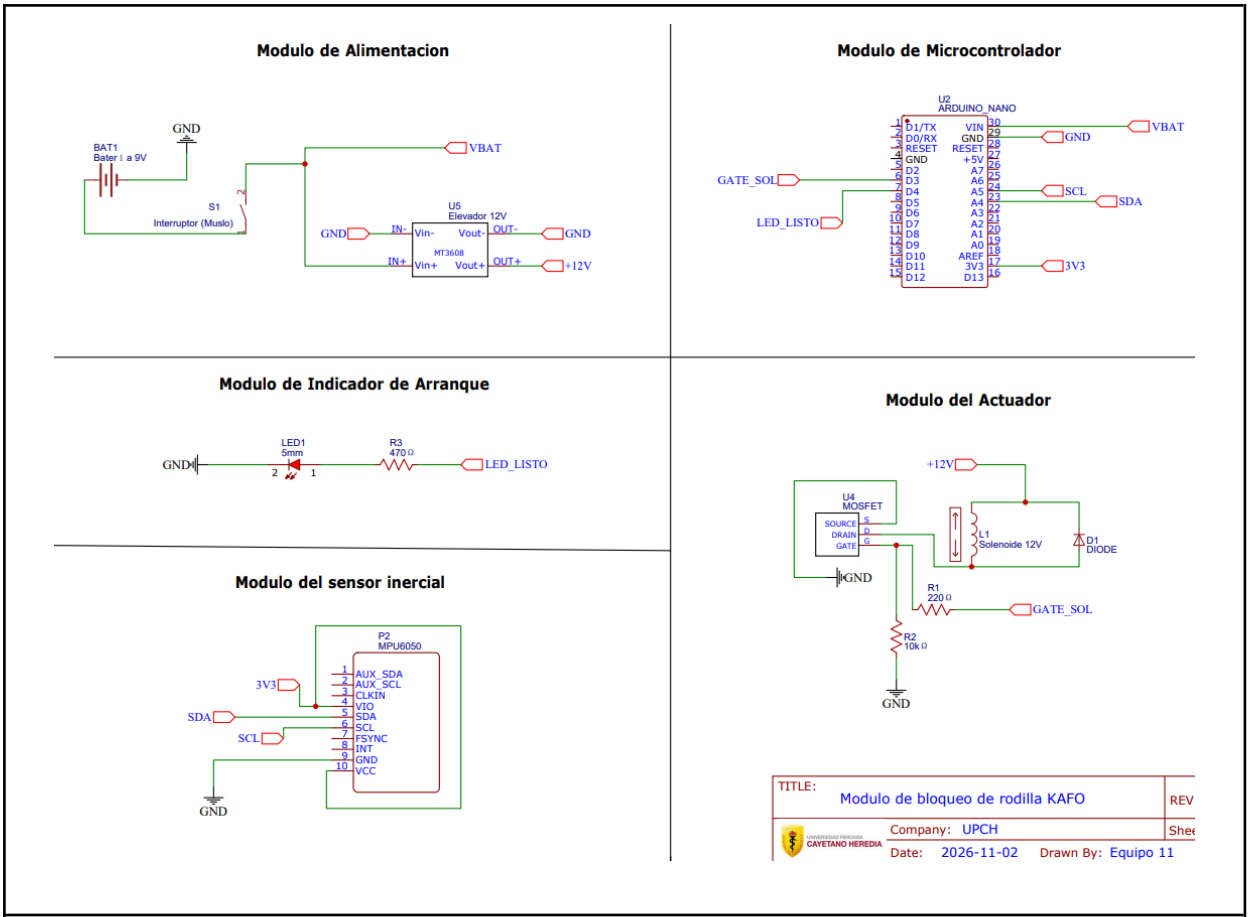
Piezas fabricadas de acuerdo con los modelos CAD y las especificaciones del diseño. Fotografías de las piezas fabricadas y comparación con el modelo CAD. MDF 5,5 mm.

1.2. Electrónica y sensores


LED
MODULO BOOST


SENSOR
IMU
MOSFET
ARDUINO
NANO


SENSOR
IMU
RESISTENCIAS
MODULO BOOST
ARDUINO
NANO
CABLE
USB
LED
DIODO
CONECTOR CLIP
DE BATERIA
MOSFET
SOLENOIDE
SWITCH DE
BALANCIN
BATERIA



Circuitos electrónicos montados y componentes conectados de acuerdo con el esquemático eléctrico. Fotografías del montaje electrónico, identificación de componentes y conexiones.

1.3. Programación y control

1.3.1. Configuración global del sistema

```
#include <Wire.h>

// ---------- Pines ----------
const uint8_t PIN_SOLENOIDE = 3;
const uint8_t PIN_LED       = 4;

// ---------- MPU6050 ----------
const uint8_t MPU_DIR = 0x68;   // dirección I2C (AD0 sin conectar o a GND)

// ---------- Parámetros ajustables ----------
const float ANGULO_LIBERAR    = 20.0;   // grados de elevación para liberar
const float ANGULO_REENGANCHE = 8.0;    // grados para considerar "pierna recta"
const unsigned long T_CONFIRMAR_LIBERAR    = 100;   // ms sostenido para liberar
const unsigned long T_CONFIRMAR_REENGANCHE = 150;   // ms sostenido para reenganchar
const unsigned long T_MAX_SOLENOIDE        = 8000;  // ms máximo con solenoide activo
const unsigned long PERIODO_MUESTREO       = 10;    // ms (100 Hz)
const float ALFA = 0.2;                 // filtro (0.1 = más suave, 0.5 = más rápido)
const uint8_t MAX_FALLOS = 5;           // lecturas fallidas seguidas permitidas

#define DEBUG 1   // 1 = imprime el ángulo por el monitor serie (115200)

// ---------- Variables de estado ----------
float anguloRef  = 0;        // ángulo con la pierna recta (calibración)
float anguloFilt = 0;        // ángulo relativo filtrado
bool  liberado   = false;    // true = solenoide activo
bool  armado     = true;     // false = debe volver a la extensión antes de liberar de nuevo
unsigned long tCond = 0;     // instante en que empezó la condición actual
unsigned long tLiberado = 0; // instante en que se activó el solenoide
uint8_t fallos = 0;
```

Configuración global del sistema: Se incluye la biblioteca Wire para la comunicación I2C con el sensor MPU6050, se asignan los pines del Arduino Nano (D3 para la compuerta del MOSFET que controla el solenoide y D4 para el LED de arranque) y se definen los parámetros ajustables: ángulo de liberación (20°), ángulo de reenganche (8°), tiempos de confirmación, tiempo máximo del solenoide (8 s), periodo de muestreo (10 ms) y constante del filtro. También se declaran las variables de estado que registran si la rodilla se encuentra liberada o bloqueada.

1.3.2. Funciones de lectura, calibración y seguridad

```ino
// ---------- Funciones ----------
void solenoide(bool activo) {
  digitalWrite(PIN_SOLENOIDE, activo ? HIGH : LOW);
}

// Lee el acelerómetro y calcula el ángulo del muslo respecto a la vertical.
// Se asume el eje X del MPU6050 apuntando a lo largo del muslo (hacia abajo).
bool leerAngulo(float &ang) {
  Wire.beginTransmission(MPU_DIR);
  Wire.write(0x3B);                          // registro ACCEL_XOUT_H
  if (Wire.endTransmission(false) != 0) return false;
  if (Wire.requestFrom(MPU_DIR, (uint8_t)6) != 6) return false;

  uint8_t hi, lo;
  hi = Wire.read(); lo = Wire.read(); int16_t ax = (hi << 8) | lo;
  hi = Wire.read(); lo = Wire.read(); int16_t ay = (hi << 8) | lo;
  hi = Wire.read(); lo = Wire.read(); int16_t az = (hi << 8) | lo;

  float yz = sqrt((float)ay * ay + (float)az * az);
  ang = atan2(yz, (float)ax) * 180.0 / PI;  // 0° con el muslo vertical
  return true;
}

// Falla del sensor: solenoide apagado (rodilla bloqueada) y LED parpadeando rápido.
void modoFalla() {
  solenoide(false);
  liberado = false;
  while (true) {
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    delay(100);
  }
}

void calibrar() {
  float suma = 0, a;
  for (uint8_t i = 0; i < 50; i++) {
    if (!leerAngulo(a)) modoFalla();
    suma += a;
    if (i % 10 == 0) digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    delay(20);
  }
  anguloRef = suma / 50.0;
```

Funciones de lectura, calibración y seguridad: La función “leerAngulo” consulta por I2C las aceleraciones del MPU6050 en los tres ejes y calcula la inclinación del muslo respecto a la vertical mediante la función arco tangente; devuelve error si el sensor no responde. La función calibrar promedia 50 lecturas para fijar el ángulo de referencia (pierna recta), de modo que el movimiento se mide como la elevación neta del muslo. La función solenoide activa o desactiva el MOSFET, y la función “modoFalla” apaga el solenoide, dejando la rodilla bloqueada, y hace parpadear rápidamente el LED cuando el sensor falla.

1.3.3. Inicialización del sistema

```ino
void setup() {
  pinMode(PIN_SOLENOIDE, OUTPUT);
  solenoide(false);                 // arranca bloqueado
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, LOW);

  Serial.begin(115200);
  Wire.begin();
#if defined(WIRE_HAS_TIMEOUT)
  Wire.setWireTimeout(3000, true);  // evita que se cuelgue si el sensor se desconecta
#endif

  // Despertar el MPU6050 (sale del modo sleep)
  Wire.beginTransmission(MPU_DIR);
  Wire.write(0x6B);
  Wire.write(0x00);
  if (Wire.endTransmission() != 0) modoFalla();
  delay(100);

  calibrar();
  anguloFilt = 0;
  digitalWrite(PIN_LED, HIGH);      // LED fijo = listo
}
```

Inicialización del sistema: Al encender el dispositivo, el solenoide se deja apagado para que la rodilla inicie bloqueada. Luego se inicia la comunicación I2C, se despierta el sensor MPU6050 y se ejecuta la calibración. Finalmente, el LED de arranque queda encendido de forma fija para indicar que el sistema está listo.

1.3.4. Funcionamiento del programa principal

```
void loop() {
  static unsigned long tUlt = 0;
  unsigned long ahora = millis();
  if (ahora - tUlt < PERIODO_MUESTREO) return;
  tUlt = ahora;

  // 1) Leer sensor
  float a;
  if (!leerAngulo(a)) {
    if (++fallos >= MAX_FALLOS) {   // sensor perdido: bloquear
      solenoide(false);
      liberado = false;
      tCond = 0;
      digitalWrite(PIN_LED, (ahora / 100) % 2);   // parpadeo = falla
    }
    return;
  }
  fallos = 0;
  digitalWrite(PIN_LED, HIGH);

  // 2) Ángulo relativo y filtrado
  float rel = a - anguloRef;
  anguloFilt = ALFA * rel + (1.0 - ALFA) * anguloFilt;

#if DEBUG
  static unsigned long tPrint = 0;
  if (ahora - tPrint >= 100) {
    tPrint = ahora;
    Serial.print(anguloFilt);
    Serial.print('\t');
    Serial.println(liberado ? 1 : 0);
  }
#endif
```

```
#endif

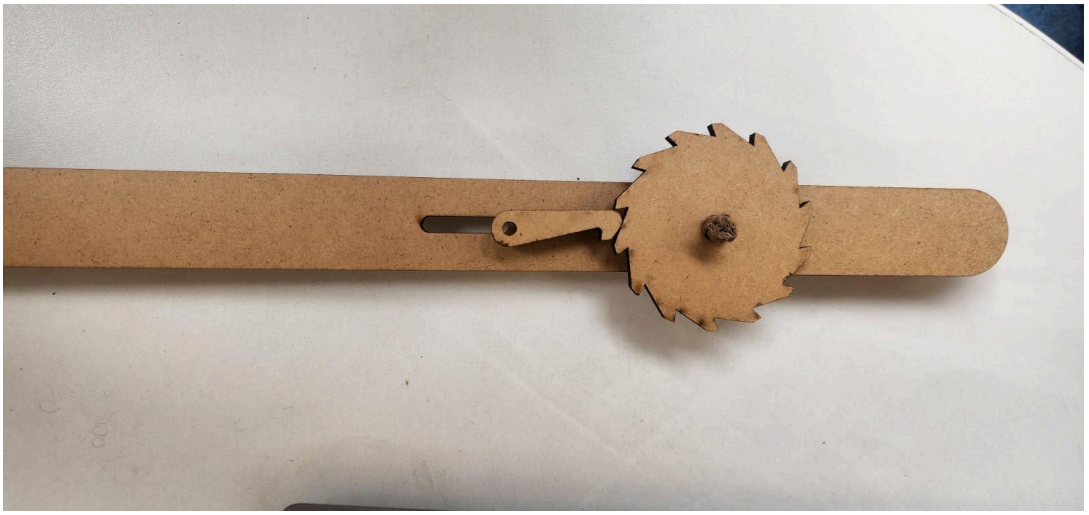
  // 3) Lógica de liberar / reenganchar
  if (!liberado) {
    if (!armado) {
      // tras un corte por tiempo, esperar a que la pierna vuelva a la extensión
      if (anguloFilt < ANGULO_REENGANCHE) armado = true;
    } else if (anguloFilt > ANGULO_LIBERAR) {
      if (tCond == 0) tCond = ahora;
      if (ahora - tCond >= T_CONFIRMAR_LIBERAR) {
        solenoide(true);
        liberado = true;
        tLiberado = ahora;
        tCond = 0;
      }
    } else {
      tCond = 0;
    }
  } else {
    if (anguloFilt < ANGULO_REENGANCHE) {
      if (tCond == 0) tCond = ahora;
      if (ahora - tCond >= T_CONFIRMAR_REENGANCHE) {
        solenoide(false);
        liberado = false;
        tCond = 0;
      }
    } else {
      tCond = 0;
    }

    // Límite de tiempo del solenoide (protege la bobina y la batería)
    if (liberado && (ahora - tLiberado > T_MAX_SOLENOIDE)) {
      solenoide(false);
      liberado = false;
      armado = false;
      tCond = 0;
    }
  }
}
```

Funcionamiento del programa principal: Cada 10 ms el sistema lee el ángulo del muslo, le resta la referencia y lo suaviza con un filtro de promedio ponderado para evitar activaciones por vibraciones. Si el solenoide está apagado y el muslo se eleva más de 20° durante 100 ms, se activa el solenoide y la rodilla queda liberada para flexionarse. Si el muslo regresa a menos de 8° durante 150 ms, el solenoide se desactiva y el resorte vuelve a bloquear la rodilla. El uso de dos umbrales distintos evita activaciones repetidas cuando el ángulo está cerca del límite. Como medidas de seguridad, si el sensor falla cinco veces seguidas el solenoide se apaga, y si permanece activo más de 8 s se desactiva automáticamente.

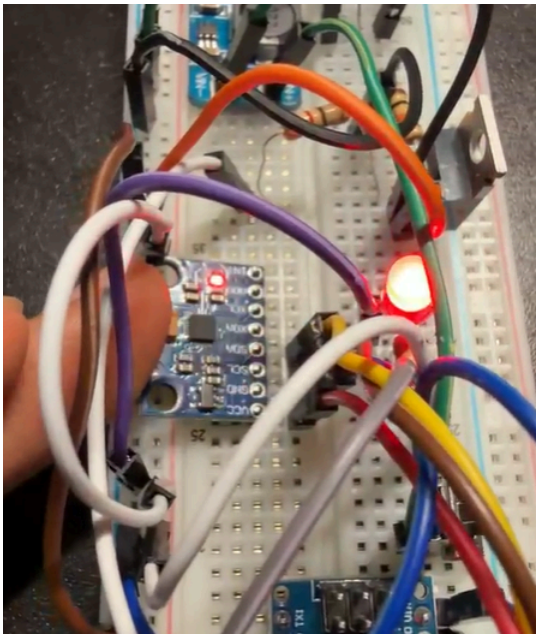
2. Verificación preliminar del diseño por subsistemas

2.1. Mecánica y fabricación



Se comprobó que ambas piezas podían posicionarse adecuadamente y que la uña alcanzaba los dientes de la rueda dentada. Asimismo, se verificó que, cuando el solenoide se encuentra desactivado (bloqueado), la disposición de los dientes permite el movimiento de la articulación en dirección a la extensión, mientras que restringe su movimiento en sentido contrario. De esta manera, el mecanismo permite que la rodilla alcance su posición de extensión completa para recuperar el bloqueo, sin necesidad de volver a activar el solenoide.

2.2. Electrónica y sensores



Prueba preliminar del circuito electrónico para verificar su encendido y la respuesta del indicador LED. Se comprobó el encendido del circuito y del indicador LED; sin embargo, se presentaron dificultades en la comunicación entre el sensor MPU6050 y el Arduino Nano, lo que impidió obtener lecturas de inclinación para evaluar su funcionamiento. Se identificó como posible causa la falta de soldadura en los pines de conexión del módulo MPU6050, lo que

podría ocasionar contactos eléctricos inestables. Además, no se logró activar el solenoide durante las pruebas realizadas, posiblemente debido a una corriente insuficiente suministrada por el sistema de alimentación.

2.3. Programación y control

Se desarrolló el programa de control para el Arduino Nano, estableciendo los parámetros de funcionamiento del mecanismo electromecánico. Se definieron umbrales de inclinación de 20° para la activación del solenoide y 8° para su desactivación, junto con tiempos de confirmación de 100 ms y 150 ms, respectivamente. Estos valores se establecieron como parámetros preliminares, donde el umbral de 20° busca distinguir una elevación significativa del muslo de pequeñas variaciones de inclinación, mientras que el de 8° permite identificar su retorno hacia la posición de referencia. La diferencia entre ambos umbrales introduce una histéresis de 12°, con el objetivo de evitar activaciones y desactivaciones repetidas del actuador ante pequeñas oscilaciones de la señal.

Debido a las dificultades de comunicación entre el sensor MPU6050 y el Arduino Nano, no fue posible verificar experimentalmente la respuesta del programa ante los cambios de inclinación. Adicionalmente, la falta de activación del solenoide impidió comprobar la ejecución física de las órdenes de control.